

Implementing an Inverse Fourier Transform in JUCE

Goal of implementation is to analyse an audio source and then resynthesize the audio with ability to control parameters.

- Using the previously built FFT to analyse a short audio source into complex numbers and then use these complex numbers to perform the inverse operation and playback the sound.
- An extra addition could be to save the new resynthesized audio clip.
- For this system we would have to utilise multiple Short-Time-Fourier-Transforms that overlap so that the audio played back is realistic/changing and not a constant tone.
- To keep the computational cost down the project will take a 5 Second audio recording with multiple small STFTs to analyse the audio source , when rebuilding the audio a windowing function would need to be applied on the ends of the the Audio signals to reduce audio discontinuities.
- To apply the multiple small STFTs, we slice the 5 second audio into overlapping windows of audio , then apply a windowing function to each , analyse the audio and store the data in arrays of complex numbers , we then take the complex numbers that store the magnitude and phase of the frequency bins. We then perform the IDTF on each window , apply a windowing function to each of the short audio clips and play them back overlapping each other to have a continuous sound source.

- The equation for IDFT is : $x[n] = \frac{1}{N} \sum_k X[k] \cdot e^{+i2\pi kn/N}$

- The equation for DFT is : $X[k] = \sum_n x[n] \cdot e^{-i2\pi kn/N}$

- In the DFT the e exponent is used to extract the magnitude and phase of a specific frequency in a wave by plotting all the points with the weights and measuring its phase relation and magnitude. In the IDFT the e exponent takes the magnitude and phase that we know and along with the data of $X[k]$ generates a sine wave using the values.

- After performing the IDFT you get returned samples of audio after you pass in the array of complex numbers.
- To implement an IDFT you use a more efficient method called the IFFT , the interesting part is that the IFFT is almost identical to the process of performing the FFT so we can reuse the majority of the function, the only two changes needed are the sign of the angle and the scaling at the end , additionally the function would need to take a parameter of complex numbers and return a parameter of floats.
- To compute the IFFT on a sample of complex numbers you first split the data into odd and even samples and recursively split until you reach the base case of a single sample being passed through , at this point you return the sample itself, after you reached this point the function continues to use butterfly calculations on the way back up using the positive angle instead of the negative angle in the FFT , the final result of this step is a vector of complex numbers the same size as the original complex numbers but now they represent a sample in the time domain , rather than a magnitude and phase value in the frequency domain , as it data in the time domain all the .imaginary numbers cancel out in the final phase so for the implementation of the final phase we ignore them. The final step is turning complex numbers that represent time domain data into float values that can be used to play back the audio, we loop through each of the results of the previous stage take the real result and save the data into a vector of floats to be used in playback.
- In the FFT , the bins are mirrored upper half mirrors the bottom half , but after converting back to the time domain the samples are not mirrored
- The reason that the imaginary values are canceled is due to the fact that the inputted samples into the original FFT are real-world valued , therefore the FFT upper half is a conjugate mirror of the bottom half.
- For completing the project of taking the x number of highest magnitude bins and converting them back to time domain to play back, after completing the FFT we need to store the full FFT bin , then find the x largest magnitudes in the FFT bin of size half , to remove the frequency bins we don't want from the whole FFT bins we need to set their value to $0 + 0i$ meaning they have no effect on the end signal but are still there to rebuild the signal correctly. Additionally if we decide to keep k , $k + \frac{fftsize}{2}$ must also be kept due to the symmetry of the Discrete Fourier Transformation. We then pass this updated vector of complex numbers into the IFFT function to get the samples back out.

- Additional care needs to be taken at the point at which each sample is played back and the overlapping of each of the splits , windowing needs to be applied to reduce clicks and pops from the final result.
- Choosing the number of FFTS and therefor IFFTS to use for a 5 second clip so that they slightly overlap , using a size of 2048 bins per transform.

$$numberOfWindows = \frac{totalSamples}{hopSize} \text{ and ,}$$

$totalSamples = sampleRate \cdot 5$. If we run at a sample rate of 44100 and a overlap of 50% for a use of hann windowing function to ensure that the amplitude remains level , that means then that the total number of windows needed to have full coverage of a 5 second clip would be 216 windows.

- To get the position of the mirror of bin k , you use the equation $N - k$, where N is the FFT size.